



第十章 Numpy数值计算

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

1. Numpy概述
2. NumPy基础
3. 数组运算
4. 逻辑运算与条件筛选
5. 排序、搜索与计数
6. Numpy的向量化
7. 案例：线性回归分析



1. Numpy概述

- **数值计算：**

- 在现代科学和工程计算中，处理大规模数据和执行复杂的数值计算已成为常态。
- Python作为一种高效、易学的编程语言，其在数据科学领域的广泛应用部分归功于NumPy库

- **NumPy的主要功能：**

- **NumPy (Numerical Python)**：Python编程语言的一个核心库，专为处理大规模的数值数据而设计，其主要功能包括：
- **多维数组对象 (ndarray)**：提供对多维数组的高效操作。
- **数值计算的高性能函数**：支持快速的数学运算，包括线性代数、傅里叶变换等。
- **强大的广播机制**：允许对不同形状的数组进行灵活的运算。

- **NumPy的优势体现在：**

- **高效的数值计算**：NumPy的核心是其多维数组对象，提供了快速、灵活的数组操作，这在处理大型数据集时尤为重要。
- **Python科学计算的基础**：NumPy是许多其他科学计算库（如SciPy、Pandas、Matplotlib、TensorFlow等）的基础，这些库广泛用于数据分析、机器学习和人工智能领域。
- **跨语言性能优势**：NumPy在底层使用C和Fortran编写，以确保性能接近于纯C或Fortran程序，同时保留了Python的灵活性和易用性。



2. NumPy基础

• 2.1 NumPy的安装与配置

- 在开始使用NumPy之前，需要先进行安装和配置。NumPy可以通过Python包管理工具pip进行安装。
- 要使用pip安装NumPy，打开命令行或终端，输入以下命令：
 - `pip install numpy`
- 安装NumPy后，通过Python解释器导入NumPy来验证安装是否成功：
 - `import numpy as np`
 - `print(np.__version__)`
 - 执行上述代码，如果输出了NumPy的版本号，则表示安装成功。



2.2 ndarray对象

- **ndarray (N-dimensional array)** : 是NumPy的核心数据结构, 支持多维数组和矩阵运算。与Python的列表list相比, ndarray具有更高的性能和更丰富的功能。创建ndarray有两种方法。
 - **方法一：从Python列表或元组创建**：NumPy可将Python的列表或元组转换为ndarray对象：
 - `import numpy as np`
 - `# 从列表创建一维数组`
 - `array_1d = np.array([1, 2, 3, 4, 5])`
 - `print(array_1d)`
 - `# 从嵌套列表创建二维数组`
 - `array_2d = np.array([[1, 2, 3], [4, 5, 6]])`
 - `print(array_2d)`



◦ **方法二：使用NumPy内置函数：** NumPy提供了多种内置函数用于创建特定类型的数组。

- (1) **np.zeros(shape):** 创建全零数组。

- `zeros_array = np.zeros((3, 3))`
 - `print(zeros_array)`

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
```

- (2) **np.ones(shape):** 创建全一数组。

- `ones_array = np.ones((2, 4))`
 - `print(ones_array)`

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
```

- (3) **np.eye(N):** 创建N×N的单位矩阵。

- `eye_array = np.eye(4)`
 - `print(eye_array)`

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]
```

- (4) **np.arange(start, stop, step):** 用于创建一个等差数列的数组，从 start 开始（包含）到 stop 结束（不包含），以 step 为步长。

- `arange_array = np.arange(0, 10, 2)`
 - `print(arange_array)`

```
[0 2 4 6 8]
```

- (5) **np.linspace(start, stop, num):** 用于创建一个等间隔数列的数组，从 start 开始到 stop 结束（包含），生成的数组具有 num 个元素。linspace方法名称来源于linear space（线性空间）的缩写。

- `linspace_array = np.linspace(0, 1, 5)`
 - `print(linspace_array)`

```
[0.    0.25 0.5   0.75 1.   ]
```



2.3 ndarray的属性和方法

- NumPy数组对象具有丰富的属性和方法，用于获取数组的基本信息和执行常见操作：
 - **ndarray的属性**
 - `ndarray.shape`: 数组的形状
 - `ndarray.size`: 数组的元素总数
 - `ndarray.dtype`: 数组元素的数据类型
 - `ndarray.ndim`: 数组的维数
 - `array = np.array([[1, 2, 3], [4, 5, 6]])`
 - `print(array.shape)` # 输出: (2, 3)
 - `print(array.size)` # 输出: 6
 - `print(array.dtype)` # 输出: int64 (根据系统不同, 可能不同)
 - `print(array.ndim)` # 输出: 2

◦ ndarray的方法

- **ndarray.reshape(shape)**: 改变数组形状，将一个数组的形状从一个维度变换为另一个维度，其中shape用一个整数元组表示。该方法按照元素的行优先顺序（即按行主序，也叫C-order）来重新排列数组的形状。这意味着元素在内存中的存储顺序不会改变，而是将这些元素按照新的形状进行重新解释。

- `array = np.array([[1, 2, 3], [4, 5, 6]])`

- `reshaped_array = array.reshape((3, 2))`

- `print(reshaped_array)`

```
[[1 2]
 [3 4]
 [5 6]]
```

与AI协作探索：Numpy中，什么是F-order，它有什么作用？

- **ndarray.flatten()**: 将多维数组展开为一维。

- `flattened_array = array.flatten()`

- `print(flattened_array)`

```
[1 2 3 4 5 6]
```

- **ndarray.transpose()**: 转置数组。

- `transposed_array = array.transpose()`

- `print(transposed_array)`

```
[[1 4]
 [2 5]
 [3 6]]
```

- **ndarray.astype(dtype)**: 转换数组的数据类型，返回一个新数组，数组的数据类型为指定的类型，其中dtype参数指定要转换成的数据类型。

- `float_array = array.astype(float)`

- `print(float_array)`

```
[[1. 2. 3.]
 [4. 5. 6.]]
```




2.4 数组的索引和切片

- NumPy支持丰富的索引和切片操作，允许对数组进行灵活的访问和修改。
 - **(1) 基本索引：**使用整数索引访问和修改数组中的单个元素。
 - `array = np.array([10, 20, 30, 40, 50])`
 - `print(array[0])` # 输出：10
 - `array[1] = 25`
 - `print(array)` # 输出：[10 25 30 40 50]
 - **(2) 切片操作：**使用切片访问数组的子集，语法类似于 Python 的 list，但在 NumPy 中切片返回的是视图 (view) 而非副本，这意味着对切片的修改会直接影响原数组。NumPy 支持多维切片，允许在每个维度上进行灵活的切片操作。
 - `array = np.array([10, 20, 30, 40, 50])`
 - `sub_array = array[1:4]`
 - `print(sub_array)` # 输出：[20 30 40]
 - # 二维数组切片
 - `array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`
 - `sub_array_2d = array_2d[:2, 1:]`
 - `print(sub_array_2d)` # 输出：[[2 3] [5 6]]

思考：如何通过切片操作取出 [[5,6], [8,9]]?



- **(3) 布尔索引：**根据条件表达式直接从数组中提取满足特定条件的元素。具体来说，布尔索引是利用布尔数组对原数组进行索引的一种方法。布尔数组与原数组形状相同，其中的布尔值 (True 或 False) 用于指示是否选择对应位置的元素。当布尔值为 True 时，选择该位置的元素；否则，忽略该元素。

- `import numpy as np`
- `arr = np.array([1, 2, 3, 4, 5])`
- `# 创建一个布尔索引条件，生成布尔数组`
- `bool_index = arr > 3`
- `print(bool_index)`

运行结果：

`[False False False True True]`

- 可以使用布尔索引从数组中选择元素。
- `# 使用布尔索引从数组中选择元素`
- `filtered_arr = arr[bool_index]`
- `print(filtered_arr)`

运行结果：

`[4 5]`

思考：如何选择所有 ≥ 4 或 ≤ 2 的元素？

- 在上例中，布尔索引条件是通过**布尔条件表达式**来生成布尔数组，用于筛选原数组中满足特定条件的元素。这个布尔条件表达式可以是任何返回布尔值的比较操作，比如等于 (`==`)、大于 (`>`)、以及组合多个条件的逻辑操作符 (`&`, `|`, `~`) 等。例如，布尔表达式 `arr > 3` 会遍历 `arr` 数组的每个元素，并对每个元素返回相应的布尔值 True 或 False，这与 `map()` 函数对每个元素应用一个函数非常相似。



3. 数组运算

- NumPy提供了丰富的数组运算功能，支持各种数学运算、统计分析和线性代数计算
- **(1) 基本数学运算**
 - NumPy中的数组运算允许用户在数组间执行各种数学操作，而无需显式编写循环。这些运算支持标量与数组间的运算以及数组间的元素级运算。
 - **(1) 标量与数组运算**：NumPy支持对数组中的每个元素执行标量运算。例如，数组可以与标量相加、相减、相乘或相除，这些运算将应用于数组的每个元素。
 - `import numpy as np`
 - `# 创建一个数组`
 - `array = np.array([1, 2, 3, 4])`
 - `# 标量加法`
 - `result_add = array + 10`
 - `print(result_add) # 输出: [11 12 13 14]`
 - `# 标量乘法`
 - `result_mul = array * 2`
 - `print(result_mul) # 输出: [2 4 6 8]`



3. 数组运算

- **(2) 数组间运算**: 对于数组间的运算, NumPy支持逐元素的加、减、乘、除。这要求参与运算的两个数组具有相同的形状。
- `array1 = np.array([1, 2, 3])`
- `array2 = np.array([4, 5, 6])`
-
- `# 数组加法`
- `result_sum = array1 + array2`
- `print(result_sum) # 输出: [5 7 9]`
-
- `# 数组乘法`
- `result_prod = array1 * array2`
- `print(result_prod) # 输出: [4 10 18]`



- **(3) 广播机制**: 允许不同形状的数组之间执行运算, 前提是它们的形状在某些维度上兼容。广播机制会自动扩展较小的数组, 使其与较大的数组匹配。
- `array = np.array([[1, 2, 3], [4, 5, 6]])`
- `scalar = 10` [[11 12 13]
[14 15 16]]
- `# 广播机制将标量扩展为数组形状`
- `result_broadcast = array + scalar`
- `print(result_broadcast)`
- 如果两个数组的形状在最后几维相同, 或者其中一个数组的维度为1, 则可以执行广播。
- `array1 = np.array([[1, 2, 3], [4, 5, 6]]) # 形状(2, 3)`
- `array2 = np.array([[10, 20, 30]]) # 形状(1, 3)`
- `result_broadcast = array1 + array2` [[11 22 33]
[14 25 36]]
- `print(result_broadcast)`
- 上例中, `array1` 的形状是 (2, 3), 而 `array2` 的形状是 (1, 3)。由于它们的形状在最后一维相同, 且 `array2` 的第一维是 1, 根据广播规则, `array2` 可以在计算时沿第一个维度自动扩展, 变成形状 (2, 3), 即 `[[10, 20, 30], [10, 20, 30]]`, 从而与 `array1` 相加得到结果。



- **(2) 统计函数：** NumPy提供了多种统计函数，用于分析数组数据的分布特征。这些函数可以用于计算数组的均值、标准差、方差、最大值、最小值等。
 - **1) 均值、标准差、方差**
 - **均值 (mean)：** 数组元素的平均值。
 - `array = np.array([1, 2, 3, 4, 5])`
 - `mean_value = np.mean(array)`
 - `print(mean_value) # 输出: 3.0`
 - **标准差 (standard deviation)：** 描述数据的离散程度。
 - `std_deviation = np.std(array)`
 - `print(std_deviation) # 输出: 1.4142135623730951`
 - **方差 (variance)：** 标准差的平方，表示数据分布的离散程度。
 - `variance_value = np.var(array)`
 - `print(variance_value) # 输出: 2.0`
- 这些函数可以通过指定axis参数来沿数组的特定轴进行计算。在NumPy中，axis参数用于指定数组操作的方向，它决定了在哪个方向上压缩数组。以二维数组为例：
 - **axis=0:** 沿着列的方向（垂直方向）进行操作，跨行进行运算，对于axis = 0，操作会在每一列上进行，这意味着结果是对每一列的值进行聚合。
 - **axis=1:** 沿着行的方向（水平方向）进行操作，跨列进行运算。对于axis = 1，操作会在每一行上进行，这意味着结果是对每一行的值进行聚合。
 - `array_2d = np.array([[1, 2, 3], [4, 5, 6]])`
 - **# 沿列计算均值**
 - `mean_along_axis0 = np.mean(array_2d, axis=0)`
 - `print(mean_along_axis0) # 输出: [2.5 3.5 4.5]`
 - **# 沿行计算标准差**
 - `std_along_axis1 = np.std(array_2d, axis=1)`
 - `print(std_along_axis1) # 输出: [0.81649658 0.81649658]`



◦ 2) 最大值、最小值

- 最大值 (max) : 数组中的最大元素。

- `max_value = np.max(array)`
 - `print(max_value)` # 输出: 5

- 最小值 (min) : 数组中的最小元素。

- `min_value = np.min(array)`
 - `print(min_value)` # 输出: 1

- 同样, `axis`参数可以用于在特定轴上计算最大值和最小值:

- # 沿列计算最大值
 - `max_along_axis0 = np.max(array_2d, axis=0)`
 - `print(max_along_axis0)` # 输出: [4 5 6]
 - # 沿行计算最小值
 - `min_along_axis1 = np.min(array_2d, axis=1)`
 - `print(min_along_axis1)` # 输出: [1 4]



• (3) 线性代数运算:

- **(1) 矩阵乘法:** 使用`np.dot()`函数或`@`运算符进行矩阵乘法
- `matrix1 = np.array([[1, 2], [3, 4]])`
- `matrix2 = np.array([[5, 6], [7, 8]])`
- `# 矩阵乘法`
- `product = np.dot(matrix1, matrix2)`
- `# product = matrix1 @ matrix2 # 同理`
- `print(product)`
- 运行结果:
- `[[19 22]`
- `[43 50]]`

- **(2) 逆矩阵与特征值:** 矩阵A的逆矩阵是一个矩阵B, 使得 $AB=I$ (单位矩阵), 计算逆矩阵需要确保矩阵是方阵且可逆。
NumPy 库中 `numpy.linalg` 模块提供的函数`inv()`用于计算矩阵的逆矩阵。
- `from numpy.linalg import inv`
- `matrix = np.array([[1, 2], [3, 4]])`
- `inverse_matrix = inv(matrix)`
- `print(inverse_matrix)`
- 运行结果:
- `[[ -2. 1.]`
- `[ 1.5 -0.5]]`



- **(3) 特征值与特征向量:** 特征值是线性变换不变的标量值, 特征向量是在变换下保持方向不变的向量。使用`np.linalg.eig()`计算特征值和特征向量。
- `from numpy.linalg import eig`
- `matrix = np.array([[1, 2], [2, 1]])`
- `eigenvalues, eigenvectors = eig(matrix)`
- `print("特征值:", eigenvalues)`
- `print("特征向量:\n", eigenvectors)`
- 运行结果:
- 特征值: [3. -1.]
- 特征向量:
- `[[ 0.70710678 -0.70710678]`
- `[ 0.70710678 0.70710678]]`



4. 逻辑运算与条件筛选

- NumPy支持灵活的逻辑运算与条件筛选功能，允许用户基于条件高效地过滤和操作数组数据。
- (1) 布尔数组**：是由布尔值（True或False）组成的数组，用于逻辑运算和条件筛选。布尔数组通常由比较操作生成，
 - # 创建一个数组
 - `array = np.array([1, 2, 3, 4, 5])`
 - # 生成布尔数组
 - `bool_array = array > 3`
 - `print(bool_array)` # 输出: [False False False True True]
 - 布尔数组可以用于数组的元素级逻辑运算
 - `array1 = np.array([True, False, True])`
 - `array2 = np.array([False, False, True])`
 - # 布尔与
 - `result_and = np.logical_and(array1, array2)`
 - `print(result_and)` # 输出: [False False True]
 - # 布尔或
 - `result_or = np.logical_or(array1, array2)`
 - `print(result_or)` # 输出: [True False True]
- (2) 条件索引与过滤**：使用布尔数组对数组进行条件索引和过滤，从数组中提取满足特定条件的元素。
 - `array = np.array([10, 20, 30, 40, 50])`
 - # 使用布尔索引过滤数组元素
 - `filtered_array = array[array > 30]`
 - `print(filtered_array)` # 输出: [40 50]
 - 在此例中，`array > 30` 会返回一个布尔数组 [False, False, False, True, True]，表示每个元素是否大于 30。使用 `array[array > 30]`，利用布尔索引的方法只提取布尔数组中值为 True 的位置对应的元素。
 - 条件筛选**也可以结合其他操作进行更复杂的数据处理。
 - `array = np.array([[1, 2, 3], [4, 5, 6]])`
 - # 使用条件筛选替换数组中的特定值
 - `array[array % 2 == 0] = 0`
 - `print(array)`
 - 运行结果:
 - `[[1 0 3]`
 - `[0 5 0]]`



5. 排序、搜索与计数

- NumPy提供了一系列函数用于对数组进行排序、搜索和计数操作，高效组织和分析数据。
- **(1) 数组排序**：NumPy的sort()函数用于对数组进行排序，返回排序后的数组：
 - `array = np.array([3, 1, 2, 5, 4])`
 - `# 对数组进行排序`
 - `sorted_array = np.sort(array)`
 - `print(sorted_array) # 输出: [1 2 3 4 5]`
 - NumPy也支持对多维数组的特定轴进行排序：
 - `array_2d = np.array([[10, 1, 8], [9, 5, 6]])`
 - `# 沿列排序`
 - `sorted_array_along_axis0 = np.sort(array_2d, axis=0)`
 - `print(sorted_array_along_axis0)`
 - 运行结果：
 - `[[ 9 1 6]`
 - `[10 5 8]]`
- **(2) 唯一值与去重**：NumPy提供unique()函数用于查找数组中的唯一值，并去除重复项。
 - `array = np.array([1, 2, 2, 3, 3, 3, 4])`
 - `# 查找唯一值`
 - `unique_values = np.unique(array)`
 - `print(unique_values) # 输出: [1 2 3 4]`



6. Numpy的向量化

- 向量化是 NumPy 的一个核心特性，通过**批量操作**替代**逐元素的标量运算**或**循环处理**，以显著提升性能、减少代码复杂性，并充分利用底层硬件（如 SIMD 指令、并行计算等）
- **(1) 消除显式循环**
 - 避免 Python 层面的 for 循环，减少解释器的开销。通过 NumPy 的批量操作，直接对数组整体进行计算。
 - `import numpy as np`
 - `# 循环方式`
 - `lst = [1, 2, 3, 4, 5]`
 - `result = [x * 2 for x in lst]`
 - `# 向量化方式`
 - `arr = np.array(lst)`
 - `result = arr * 2 # [2, 4, 6, 8, 10]`
- **(2) 逐元素操作的批量化**
 - 将标量的逐元素运算扩展为数组的批量操作，例如算术运算、数学函数等。
 - `# 标量方式`
 - `x = 2`
 - `result = x ** 2`
 - `# 向量化方式`
 - `arr = np.array([1, 2, 3, 4])`
 - `result = arr ** 2 # [1, 4, 9, 16]`



- **(3) 内置数学函数**

- NumPy 提供的数学函数（如 sin、cos、sqrt）可以直接应用于数组，这些函数本身是向量化的。
- `arr = np.array([0, np.pi / 2, np.pi])`
- `result = np.sin(arr) # [0.0, 1.0, 0.0]`

- **(4) 简化复杂运算逻辑**

- 通过广播、内置函数和高级索引，将复杂的多步骤操作合并为单步或少量操作。
- # 循环方式
- `arr = [1, 2, 3, 4]`
- `result = sum([x**2 for x in arr])`
- # 向量化方式
- `arr = np.array([1, 2, 3, 4])`
- `result = np.sum(arr ** 2) # 更高效`



7. 案例：线性回归分析

- **线性回归：**

- 在数据科学中，线性回归是一种常用的预测模型，通过拟合一个线性方程来描述自变量和因变量之间的线性关系。

- **例：使用线性回归模型分析并预测GDP**

- 使用线性回归模型来对模拟的GDP数据进行分析，并预测未来两年的GDP。整个过程包括数据准备、年度增长率计算、回归模型训练和未来数据的预测。



- (1) 导入库并模拟 GDP 数据

- 首先导入NumPy库，并模拟一组GDP时间序列数据作为输入。
- `import numpy as np`
- `# 模拟GDP时间序列数据（单位：万亿美元）`
- `gdp_data = np.array([18.57, 18.71, 18.83, 18.96, 19.08, 19.21, 19.33, 19.45, 19.57, 19.70])`

- (2) 计算年度增长率

- 年度增长率用于衡量经济增长情况。计算方法为相邻年份的GDP差值除以前一年的GDP。
- `# 计算GDP的年度增长率`
- `gdp_growth_rate = np.diff(gdp_data) / gdp_data[:-1] * 100`
- `print("GDP 年度增长率:", gdp_growth_rate)`
- `np.diff()` 方法用于计算数组中相邻元素的差值。它会返回一个新数组，其中的每个元素是原数组中相邻元素的差值。在此例中，`np.diff(gdp_data)` 计算了 GDP 数据中相邻年份之间的差值，用以衡量年度 GDP 的变化。



- (3) 准备回归模型的数据

- 线性回归模型通过拟合直线方程，描述自变量与因变量之间的关系。这里将年份数据作为自变量 (X)，GDP 数据作为因变量 (y)。为了进行回归分析，还需要为自变量增加偏置项 (截距)，以便模型可以更好地拟合。
- # 准备回归模型的数据，X为自变量（年份），y为因变量（GDP）
- `X = np.arange(len(gdp_data)).reshape(-1, 1)`
- `y = gdp_data`
- # 增加偏置项（截距）
- `X_b = np.c_[np.ones((X.shape[0], 1)), X]`
 - X 使用 `np.arange()` 函数生成表示年份的数据，并通过 `reshape(-1, 1)` 转换为列向量，表示 10 年的数据序列。
 - y 为前面创建的 `gdp_data`，表示 GDP 的实际数值。
 - 然后通过 `np.ones()` 生成一个全是 1 的列向量，用作偏置项，并与 X 拼接在一起，生成包含偏置项的特征矩阵 `X_b`，以确保模型中包含截距项。`np.c_[]` 是 NumPy 提供的一种方便的拼接操作，可以将多个数组沿着列方向拼接在一起。



- **(4) 计算线性回归的系数**

- 在计算回归系数时，使用了下面的线性代数的公式来求解线性回归的最佳系数，这一步计算了回归方程中的截距和斜率。公式如下：
 - $\theta = (X_b^T X_b)^{-1} X_b^T y$
 - 其中 X_b 是带有偏置项的特征矩阵、 X_b^T 是 X_b 的转置矩阵， $(X_b^T X_b)^{-1}$ 是 $X_b^T X_b$ 的逆矩阵， y 是因变量（目标值）向量， θ 是回归模型的系数向量，包括截距和回归系数。
 - # 计算线性回归的系数
 - `theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)`
 - `print("回归系数:", theta_best)`
 - 通过矩阵运算 `X_b.T.dot(X_b)` 计算自变量矩阵的转置与自身的乘积，再使用 `np.linalg.inv()` 求逆矩阵，然后将结果与 `X_b.T.dot(y)` 相乘，最终得到回归系数 `theta_best`。这些系数包括截距项和斜率，描述了 GDP 的增长趋势与年份的关系。
- 思考：如何将上述流程封装为一个类？每个实例对应一个由具体参数 θ 定义的线性模型 $\hat{y} = \theta^T x$ ，可以完成参数学习与目标预测等基本功能。



- **(5) 预测未来的 GDP**

- 通过已拟合的线性回归模型，对未来年份进行预测。只需将未来年份的数据输入回归方程，结合计算出的回归系数，即能得出未来的 GDP 预测值。
- # 预测未来GDP
- `X_new = np.array([[len(gdp_data)], [len(gdp_data) + 1]]) # 未来两年`
- `X_new_b = np.c_[np.ones((X_new.shape[0], 1)), X_new]`
- `gdp_predict = X_new_b.dot(theta_best)`
- `print("未来GDP预测:", gdp_predict)`
- 首先创建了未来两年的年份数据 X_new，并与偏置项拼接为矩阵 X_new_b。然后将该矩阵与回归系数 theta_best 相乘，得到了未来两年的 GDP 预测值。通过这种方式，利用线性回归模型可以预测未来的经济增长情况。
- 运行结果：
- GDP 年度增长率: [0.75390415 0.64136825 0.69038768 0.63291139 0.68134172 0.62467465
- 0.62079669 0.61696658 0.66428206]
- 回归系数: [18.58109091 0.12442424]
- 未来GDP预测: [19.82533333 19.94975758]



Numpy的方法与属性汇总表

类别	方法/属性	说明
数组创建	np.array()	从Python列表或元组创建数组。
	np.zeros()	创建全零数组，指定形状。
	np.ones()	创建全一数组，指定形状。
	np.arange()	创建等差序列数组，类似于range函数。
	np.linspace()	在指定范围内创建等距数值数组。
	np.eye()	创建单位矩阵。
	np.random.rand()	创建0到1之间均匀分布的随机数数组。
	np.random.randint()	创建整数随机数数组，指定范围和形状。
数组属性	ndarray.shape	返回数组的形状（行数和列数）。
	ndarray.size	返回数组的元素总数。
	ndarray.ndim	返回数组的维数。
	ndarray.dtype	返回数组元素的数据类型。
	ndarray.itemsize	返回数组中每个元素的字节大小。
数组索引和切片	ndarray[index]	根据索引访问数组元素，支持负索引。
	ndarray[start:end:step]	进行切片操作，支持步长选择。
	np.where()	根据条件返回满足条件的索引或替换值。
	np.take()	从数组中提取指定位置的元素，支持多维数组。
数组运算	np.add()	元素逐一相加。
	np.subtract()	元素逐一相减。
	np.multiply()	元素逐一相乘。
	np.divide()	元素逐一相除。
	np.power()	对每个元素求幂。
	np.exp()	计算数组中所有元素的指数。
	np.sqrt()	计算数组中所有元素的平方根。
	np.log()	计算数组中所有元素的自然对数。
统计函数	np.abs()	计算数组中所有元素的绝对值。
	np.sum()	计算数组的总和，可以指定轴（axis）求和。
	np.mean()	计算数组的均值。
	np.median()	计算数组的中位数。
	np.std()	计算数组的标准差。
	np.var()	计算数组的方差。
	np.min()	找到数组中的最小值。
	np.max()	找到数组中的最大值。
	np.cumsum()	计算数组的累积和。
	np.cumprod()	计算数组的累积积。

类别	方法/属性	说明
数组创建	np.array()	从Python列表或元组创建数组。
线性代数运算	np.dot()	计算两个数组的点积（矩阵乘法）。
	np.matmul()	矩阵乘法，支持多维矩阵相乘。
	np.linalg.inv()	计算矩阵的逆矩阵。
	np.linalg.det()	计算矩阵的行列式。
	np.linalg.eig()	计算矩阵的特征值和特征向量。
	np.transpose()	转置矩阵。
通用函数	np.sin()	计算数组中所有元素的正弦值。
	np.cos()	计算数组中所有元素的余弦值。
	np.tan()	计算数组中所有元素的正切值。
	np.floor()	对数组中所有元素向下取整。
	np.ceil()	对数组中所有元素向上取整。
逻辑运算与条件筛选	np.all()	判断数组中是否所有元素为True。
	np.any()	判断数组中是否有任一元素为True。
	np.logical_and()	逐元素执行逻辑与操作。
	np.logical_or()	逐元素执行逻辑或操作。
	np.logical_not()	逐元素执行逻辑非操作。
排序、搜索与计数	np.where()	根据条件筛选元素，返回满足条件的位置或替换值。
	np.sort()	对数组排序。
	np.argsort()	返回数组元素排序后的索引。
	np.unique()	返回数组中唯一的元素，并去重。
数组变形	np.count_nonzero()	计算数组中非零元素的数量。
	np.reshape()	改变数组的形状。
	np.ravel()	将多维数组展开成一维数组。
	np.flatten()	返回一份数组的一维拷贝。
	np.concatenate()	沿指定轴连接多个数组。
	np.hstack()	水平堆叠数组。
随机数生成	np.vstack()	垂直堆叠数组。
	np.random.rand()	生成0到1之间均匀分布的随机数数组。
	np.random.randint()	生成整数随机数数组，指定范围和形状。
	np.random.normal()	生成服从正态分布的随机数。
	np.random.choice()	从数组中随机抽取元素。



• 一、理论知识

1. 如何查看NumPy 中ndarray数组的形状和数据类型？
2. 如何在 NumPy 中实现数组的索引和切片？与 Python 内置的列表切片有何不同？举例说明。
3. 列出并简要说明 NumPy 中常用的统计函数，如 mean、sum、max、min 等。它们在数据分析中有哪些应用？
4. 简述 NumPy 中的线性代数操作，包括矩阵乘法、逆矩阵和特征值计算。查阅资料，总结分析这些操作在数据分析和机器学习中如何应用？



• 二、编程实践

1. 定义一个 5x5 的二维数组，随机生成 1 到 10 的整数，然后将数组中的最大值替换为 0。
2. 创建一个 4x4 的二维数组，并交换其中的第二行和第三行的元素。
3. 生成一个包含 100 个随机数的一维数组，并计算这些数的均值和标准差。
4. 创建一个 10x10 的数组，按列计算最大值和最小值，并将这些值打印出来。
5. 使用 NumPy 实现逻辑筛选，从一个包含 0 到 20 的一维数组中筛选出所有大于 10 的元素。
6. 实现一个函数，计算一维数组的累积和。例如，输入 [1, 2, 3, 4] 应输出 [1, 3, 6, 10]。
7. 编写一个函数，接收一个二维 NumPy 数组（矩阵）作为输入，返回该矩阵顺时针旋转 90 度后的新矩阵。



The End !